

DNS.SetSolarWindsRecordToActiveLB.v2.ps1

PowerShell script - the same decision as the original, reworked: SNMP failures are handled rather than fatal, a data centre is polled more than once before being declared down, and the run is logged, validated and previewable with -WhatIf.

411 lines · Scripts/DNS.SetSolarWindsRecordToActiveLB.v2.ps1

```
1 #Requires -Version 5.1
2
3 <#
4 .SYNOPSIS
5     Points the SolarWinds DNS A record at the load balancer of the data centre
6     that is currently able to serve it.
7
8 .DESCRIPTION
9     The script is meant to run as an alert action on the SolarWinds server that
10    has just become active. It is told which SolarWinds server it is running
11    for, polls that data centre's web servers through the local load balancer
12    over SNMP, and repoints the A record at the local load balancer while the
13    data centre is serving, or at the other data centre's load balancer once
14    every web server there has stopped answering.
15
16    Nothing is written when the record already holds the wanted address, so the
17    script is safe to run on a schedule.
18
19 .PARAMETER ActiveServer
20     The address of the SolarWinds server this run is for. It has to match either
21     PrimaryDcSolarWindsIp or SecondaryDcSolarWindsIp, otherwise the script fails
22     instead of quietly doing nothing.
23
24 .PARAMETER DnsServer
25     The DNS server to read the record from and write it back to. It has to hold
26     a writable copy of the zone. When it is left out the script walks the DNS
27     servers configured on this host and takes the first one that answers a ping.
28
29 .PARAMETER HealthyValue
30     The SNMP value that means "this web server is up". Everything else counts as
31     down. See the note at the bottom of this help before changing it.
32
33 .PARAMETER PollCount
34     How many times a data centre is polled before it is declared down. This is
35     what keeps a single bad reading from moving the record.
36
37 .PARAMETER SnmpFailureMode
38     What to do when none of the polls get an answer at all. 'Down' treats an
39     unreachable load balancer as a dead data centre and fails over, which is the
40     behaviour the original script had. 'Unknown' leaves the record alone and
41     exits with code 2, which is the safer choice if SNMP is flaky.
42
43 .PARAMETER RecordTtlSeconds
44     When greater than zero, the TTL written onto the record. A failover only
45     takes effect once the old TTL has expired everywhere, so a low value (30-60)
46     is worth setting here.
47
48 .EXAMPLE
49     .\DNS.SetSolarWindsRecordToActiveLB.v2.ps1 -ActiveServer 10.10.10.1 -WhatIf
50
51     Shows what would change without touching DNS.
52
53 .EXAMPLE
54     .\DNS.SetSolarWindsRecordToActiveLB.v2.ps1 -ActiveServer 10.10.20.1 -Verbose
55
56 .NOTES
57     Exit codes: 0 done (changed or nothing to change), 1 failed, 2 the state of
```

```

58 the data centre could not be determined and the record was left alone.
59
60 Call it with -File, not -Command:
61
62     powershell.exe -NoProfile -File DNS.SetSolarWindsRecordToActiveLB.v2.ps1 -ActiveServer 10.10.10.1
63
64 A parameter that fails validation stops the script before it runs, and only
65 -File turns that into exit code 1. Under -Command the caller reads a stale
66 $LASTEXITCODE instead, so a typo in the arguments looks like a clean run.
67
68 HealthyValue defaults to 0 because that is what the original script compared
69 against. Confirm it against the devices before relying on this in
70 production: if 0 does not in fact mean "up" on your load balancers, every
71 decision the script makes is inverted.
72 #>
73
74 [CmdletBinding(SupportsShouldProcess)]
75 param(
76     [Parameter(Mandatory, Position = 0)]
77     [ValidateNotNullOrEmpty()]
78     [string] $ActiveServer,
79
80     [ValidateNotNullOrEmpty()]
81     [string] $RecordName = 'OurSolar',
82
83     [ValidateNotNullOrEmpty()]
84     [string] $ZoneName = 'OurZone',
85
86     [string] $DnsServer,
87
88     [ValidateNotNullOrEmpty()]
89     [string] $PrimaryDcSolarWindsIp = '10.10.10.1',
90
91     [ValidateNotNullOrEmpty()]
92     [string] $SecondaryDcSolarWindsIp = '10.10.20.1',
93
94     [ValidateNotNullOrEmpty()]
95     [string] $PrimaryDcLoadBalancerIp = '10.10.10.10',
96
97     [ValidateNotNullOrEmpty()]
98     [string] $SecondaryDcLoadBalancerIp = '10.10.20.10',
99
100    [ValidateNotNullOrEmpty()]
101    [string] $PrimaryDcSnmpHost = '10.10.10.100',
102
103    [ValidateNotNullOrEmpty()]
104    [string] $SecondaryDcSnmpHost = '10.10.20.100',
105
106    [string[]] $PrimaryDcWebServerOid = @(
107        '.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.52.52.1.2.57.51',
108        '.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.52.52.1.2.57.52'
109    ),
110
111    [string[]] $SecondaryDcWebServerOid = @(
112        '.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.51.57.1.2.51.54',
113        '.1.3.6.1.4.1.1872.2.5.4.2.19.1.10.2.51.57.1.2.51.55'
114    ),
115
116    [ValidateNotNullOrEmpty()]
117    [string] $SnmpCommunity = 'public',
118
119    [int] $HealthyValue = 0,
120
121    [ValidateRange(1, 10)]
122    [int] $PollCount = 3,

```

```

123
124 [ValidateRange(0, 60)]
125 [int] $PollDelaySeconds = 5,
126
127 [ValidateSet('Down', 'Unknown')]
128 [string] $SnmppFailureMode = 'Down',
129
130 [ValidateRange(0, 86400)]
131 [int] $RecordTtlSeconds = 0,
132
133 [string] $LogPath = (Join-Path -Path $env:ProgramData -ChildPath 'SolarWinds\dns-failover.log')
134)
135
136 Set-StrictMode -Version Latest
137 $ErrorActionPreference = 'Stop'
138
139 function Write-Log {
140     param(
141         [Parameter(Mandatory)]
142         [AllowEmptyString()]
143         [string] $Message,
144
145         [ValidateSet('INFO', 'WARN', 'ERROR')]
146         [string] $Level = 'INFO'
147     )
148
149     $line = '{0:yyyy-MM-dd HH:mm:ss} [{1,-5}] {2}' -f (Get-Date), $Level, $Message
150     Write-Information -MessageData $line -InformationAction Continue
151
152     # -WhatIf:$false so that a -WhatIf run still leaves a trace of what it
153     # decided; the log is a record, never one of the changes being previewed.
154     try {
155         $directory = Split-Path -Path $script:LogPath -Parent
156         if ($directory -and -not (Test-Path -LiteralPath $directory)) {
157             New-Item -Path $directory -ItemType Directory -Force -WhatIf:$false -Confirm:$false | Out-Null
158         }
159         Add-Content -LiteralPath $script:LogPath -Value $line -Encoding UTF8 -WhatIf:$false -Confirm:$false
160     }
161     catch {
162         Write-Warning "Could not write to the log file '$script:LogPath': $($_.Exception.Message)"
163     }
164 }
165
166 function Resolve-DnsServerAddress {
167     # Walks the DNS servers configured on this host and returns the first one
168     # that answers a ping. Only adapters with IP enabled are looked at, so
169     # disconnected or disabled NICs cannot contribute a stale address.
170     [OutputType([string])]
171     param()
172
173     $adapters = Get-CimInstance -ClassName Win32_NetworkAdapterConfiguration -Filter 'IPEnabled = True'
174
175     foreach ($adapter in $adapters) {
176         foreach ($address in $adapter.DNSServerSearchOrder) {
177             if (Test-Connection -ComputerName $address -Count 1 -Quiet -ErrorAction SilentlyContinue) {
178                 Write-Log "Using DNS server $address, configured on '$($adapter.Description)'."
179                 return [string]$address
180             }
181
182             Write-Log "DNS server $address did not answer a ping, trying the next one." -Level WARN
183         }
184     }
185
186     throw 'None of the DNS servers configured on this host answered a ping. Pass -DnsServer to name one explicitly.'

```

```

187}
188
189function Get-WebServerHealth {
190    # Reads every OID once and reports how many web servers came back healthy.
191    # Answered is false only when the load balancer gave us nothing at all,
192    # which is what tells a dead data centre apart from a dead SNMP path.
193    param(
194        [Parameter(Mandatory)][string] $Snmphost,
195        [Parameter(Mandatory)][string[]] $Oid,
196        [Parameter(Mandatory)][string] $Label
197    )
198
199    $snmp = $null
200    $answered = 0
201    $healthy = 0
202
203    try {
204        $snmp = New-Object -ComObject olePrn.oleSNMP
205        $snmp.Open($Snmphost, $SnmCommunity, 2, 1000)
206
207        foreach ($singleOid in $Oid) {
208            try {
209                $raw = ([string]$snmp.Get($singleOid)).Trim()
210            }
211            catch {
212                Write-Log "$Label $singleOid could not be read: $($_.Exception.Message)" -Level WARN
213                continue
214            }
215
216            $answered++
217
218            $value = 0
219            if (-not [int]::TryParse($raw, [ref]$value)) {
220                Write-Log "$Label $singleOid returned '$raw', which is not a number. Counting it as down." -
Level WARN
221                continue
222            }
223
224            if ($value -eq $HealthyValue) {
225                $healthy++
226                Write-Log "$Label $singleOid = $value (up)."

```

```

250     Healthy = $healthy
251 }
252 }
253
254 function Get-DataCentreState {
255     # Serving as soon as one poll finds a web server up, so a single bad reading
256     # cannot move the record. Unknown when no poll got an answer at all.
257     [OutputType([string])]
258     param(
259         [Parameter(Mandatory)][string] $Snmphost,
260         [Parameter(Mandatory)][string[]] $Oid,
261         [Parameter(Mandatory)][string] $Label
262     )
263
264     $everAnswered = $false
265
266     for ($attempt = 1; $attempt -le $PollCount; $attempt++) {
267         Write-Log "Polling $Label on $Snmphost, attempt $attempt of $PollCount."
268         $health = Get-WebServerHealth -Snmphost $Snmphost -Oid $Oid -Label $Label
269
270         if ($health.Answered) {
271             $everAnswered = $true
272
273             if ($health.Healthy -gt 0) {
274                 Write-Log "$Label has $($health.Healthy) web server(s) up."
275                 return 'Serving'
276             }
277         }
278
279         if ($attempt -lt $PollCount -and $PollDelaySeconds -gt 0) {
280             Start-Sleep -Seconds $PollDelaySeconds
281         }
282     }
283
284     if (-not $everAnswered) {
285         Write-Log "$Label did not answer any of the $PollCount SNMP polls." -Level WARN
286         return 'Unknown'
287     }
288
289     Write-Log "$Label answered, and every web server is down." -Level WARN
290     return 'Down'
291 }
292
293 function Get-SolarWindsRecord {
294     param(
295         [Parameter(Mandatory)][string] $Server
296     )
297
298     $records = @(Get-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -Name $RecordName -
299                 RRType A)
300
301     if ($records.Count -eq 0) {
302         throw "Zone '$ZoneName' on $Server holds no A record named '$RecordName'."
303     }
304
305     if ($records.Count -gt 1) {
306         throw "Zone '$ZoneName' on $Server holds $($records.Count) A records named '$RecordName'. This script
307         handles a single A record, so the duplicates have to be resolved first."
308     }
309
310     return $records[0]
311 }
312
313 function Set-SolarWindsRecord {
314     # Set-DnsServerResourceRecord needs the untouched record and the edited copy

```

```

313 # as two separate objects, hence the Clone.
314 param(
315     [Parameter(Mandatory)][string] $Server,
316     [Parameter(Mandatory)][string] $TargetIp
317 )
318
319 $current = Get-SolarWindsRecord -Server $Server
320 $updated = $current.Clone()
321 $updated.RecordData.IPv4Address = [System.Net.IPAddress]::Parse($TargetIp)
322
323 if ($RecordTtlSeconds -gt 0) {
324     $updated.TimeToLive = [System.TimeSpan]::FromSeconds($RecordTtlSeconds)
325 }
326
327 Set-DnsServerResourceRecord -ComputerName $Server -ZoneName $ZoneName -OldInputObject $current -
NewInputObject $updated
328
329 $after = Get-SolarWindsRecord -Server $Server
330 $afterIp = $after.RecordData.IPv4Address.IPAddressToString
331
332 if ($afterIp -ne $TargetIp) {
333     throw "The record was written but still reads $afterIp instead of $TargetIp."
334 }
335}
336
337try {
338    Write-Log "==== Run started for active server $ActiveServer ====="
339
340    if ($ActiveServer -eq $PrimaryDcSolarWindsIp) {
341        $localName      = 'PrimaryDC'
342        $remoteName     = 'SecondaryDC'
343        $snmpHost       = $PrimaryDcSnmpHost
344        $oids           = $PrimaryDcWebServerOid
345        $localLbIp      = $PrimaryDcLoadBalancerIp
346        $remoteLbIp     = $SecondaryDcLoadBalancerIp
347    }
348    elseif ($ActiveServer -eq $SecondaryDcSolarWindsIp) {
349        $localName      = 'SecondaryDC'
350        $remoteName     = 'PrimaryDC'
351        $snmpHost       = $SecondaryDcSnmpHost
352        $oids           = $SecondaryDcWebServerOid
353        $localLbIp      = $SecondaryDcLoadBalancerIp
354        $remoteLbIp     = $PrimaryDcLoadBalancerIp
355    }
356    else {
357        throw "ActiveServer '$ActiveServer' is neither the PrimaryDC SolarWinds server
($PrimaryDcSolarWindsIp) nor the SecondaryDC one ($SecondaryDcSolarWindsIp)."
358    }
359
360    Write-Log "Running for $localName."
361
362    $dnsServerAddress = if ($DnsServer) { $DnsServer } else { Resolve-DnsServerAddress }
363
364    $record = Get-SolarWindsRecord -Server $dnsServerAddress
365    $currentIp = $record.RecordData.IPv4Address.IPAddressToString
366    Write-Log "$RecordName.$ZoneName currently points at $currentIp with a TTL of $($record.TimeToLive)."
367
368    $state = Get-DataCentreState -SnmpHost $snmpHost -Oid $oids -Label $localName
369
370    if ($state -eq 'Unknown') {
371        if ($SnmpFailureMode -eq 'Unknown') {
372            Write-Log "$localName could not be reached over SNMP and SnmpFailureMode is 'Unknown', so the
record is left at $currentIp." -Level ERROR
373            exit 2
374        }

```

```

375
376     Write-Log "$localName could not be reached over SNMP. SnmpFailureMode is 'Down', so it is treated as
a dead data centre." -Level WARN
377     $state = 'Down'
378 }
379
380 if ($state -eq 'Serving') {
381     $targetIp = $localLbIp
382     $targetName = $localName
383 }
384 else {
385     $targetIp = $remoteLbIp
386     $targetName = $remoteName
387 }
388
389 if ($currentIp -eq $targetIp) {
390     Write-Log "Nothing to do, the record already points at the $targetName load balancer ($targetIp).\"
391     exit 0
392 }
393
394 $target = "$RecordName.$ZoneName on $dnsServerAddress"
395 $action = "Repoint from $currentIp to $targetIp, the $targetName load balancer"
396
397 if (-not $PSCmdlet.ShouldProcess($target, $action)) {
398     Write-Log "WhatIf: would have repointed $RecordName.$ZoneName from $currentIp to $targetIp
($targetName load balancer).\"
399     exit 0
400 }
401
402 Set-SolarWindsRecord -Server $dnsServerAddress -TargetIp $targetIp
403 Write-Log "Repointed $RecordName.$ZoneName from $currentIp to $targetIp, the $targetName load balancer.\"
404 exit 0
405}
406catch {
407    Write-Log "Run failed: $($_.Exception.Message)" -Level ERROR
408    Write-Log $_.ScriptStackTrace -Level ERROR
409    Write-Error -Message $_.Exception.Message -ErrorAction Continue
410    exit 1
411}

```